

# A Novel Approach for Test Case Prioritization

R.Uma Maheswari<sup>1</sup>, Dr.D.JeyaMala<sup>2</sup>

<sup>1</sup> Department of computer Applications, K.L.N.College of Engineering, Sivagangai, India

<sup>2</sup> Department of computer Applications, Thiagarajar College of Engineering, Madurai, India

([uma05raj@gmail.com](mailto:uma05raj@gmail.com) <sup>2</sup>[djmcse@gmail.com](mailto:djmcse@gmail.com))

**Abstract** -- The process of verifying the modified software in the maintenance phase is called Regression Testing. The size of the regression test suite and its selection process is a complex task for regression testers because of time and budget constraints. In this research paper, new Prioritization technique based on hamming distance has been proposed. It is illustrated using an example and found that it produces good results. Average Percentage of Fault Detection (APFD) metrics and charts has been used to show the effectiveness of proposed algorithm.

**Keywords**— APFD, Fault based Test Suite prioritization, Hamming Distance

## I. INTRODUCTION

As software evolves, test engineers regression tests it to validate new features and detect whether new faults have been introduced into previously tested code. Regression testing is important, but also expensive, so many approaches for improving its cost-effectiveness have been investigated. Among these approaches test case prioritization is important. Test case prioritization techniques help engineers execute regression tests in an order that achieves testing objectives earlier in the testing process. One testing objective involves average percentage of fault detection { a measure of how quickly a test order detects faults}. An improved rate of fault detection can provide earlier feedback on the system under test, enable earlier debugging, and increase the likelihood that, if testing is prematurely halted, those test cases that offer the greatest fault detection ability in the available testing time will have been executed.

## II. TEST CASE PRIORITIZATION

The test case prioritization problem can be defined as follows:

Given:

A test suite T, PT is the set of permutations of T and a function f from PT to the set of real numbers,

Problem: find T 'C PT such that  
 $(T'' (T'' \in PT)(T' \neq T'')[f(T') \geq f(T'')]$

In this definition, PT is the set of possible prioritizations (orders) of T, and f is an objective -function that applied to any such order, yields an award value for that order.

The definition assumes that higher award values are preferred over the lower ones. There can be number of possible goals of test case prioritization. The objective of this study is to develop a test case prioritization technique that prioritizes test cases for regression testing on the basis of Hamming Distance at the fastest rate possible based on early detection of faults.

## III. LITERATURE REVIEW

Numerous prioritization techniques has been described in the research literature [1][2][3][4].To date, most proposed techniques have been code-based, relying on information relating test cases to coverage of code elements, and a first dimension along which techniques can be distinguished is in terms of the type of code elements they consider. For example, test cases can be prioritized in terms of the number of code statements, basic blocks, or functions they executed on a previous version of the software. One technique for doing this, total function coverage prioritization, simply sorts the test cases in the order of the number of functions they cover, and if multiple test cases cover the same number of functions, orders these randomly.

Then the amount of change undergone by a code element can be factored into prioritization, by Weighting the elements covered using a fault index indicating an amount of change [6]

Such a fault index can range from a complex metric incorporating various factors related to the possible incidence of regression faults [5], to a simple binary metric indicating 'changed' or 'not changed'.

Srivastava and Thiagarajan [9] present a technique for prioritizing test cases based on basic block coverage, using both feedback and change information.

Kim and Porter [8] present a technique, which they refer to as a "history-based prioritization" that utilizes information from previous testing cycles to select the test cases that must be executed for a new version of the program

Prem and Ravi [12] prioritize them based on the number of modified lines covered by the test case.

Recently, Heuristic Techniques like Genetic Algorithm, Bee Colony Optimization are used mostly for Prioritization. Sabharwal [10] has generated prioritized test case in static testing using genetic algorithm

Badwal and Raperia [11] use clustering approach for prioritization.

#### IV. PROPOSED WORK

In the proposed technique the test cases are prioritized in such a manner so as to achieve maximum fault coverage at the earliest as well as to reduce time and cost. To ensure that all the faults are covered by the test cases selected in the prioritized ordering, mutation testing is used.(i.e)program's code is deliberately altered, then re-run a suite of valid test cases against the mutated program. A good test will detect the change (fault) in the program and fail accordingly. Thus, the proposed algorithm's effectiveness is measured using mutation testing. This is illustrated by the example shown in table I

TABLE I  
TEST CASES WITH FAULTS COVERED

| Test Case | F1 | F2 | F3 | F4 | F5 |
|-----------|----|----|----|----|----|
| T1        | x  |    | x  | x  |    |
| T2        | x  |    | x  |    |    |
| T3        | x  | x  |    |    |    |
| T4        | x  |    |    |    | x  |
| T5        | x  | x  | x  |    |    |

Faults exposed by each test case are represented as a binary string. The similarity between two test cases is measured by the distance between two binary strings using Hamming distance.

The Hamming distance between two strings of equal length is the number of positions at which the corresponding symbols are different.

Example

Hamming Distance between 1011101 and 1001001 is 2.

In our proposed work, a test case of high hamming distance with the existing one is chosen at each Step until all faults are covered.

##### A. Algorithm

Inputs

T- Original Test Suite

n- No of Test Cases

m-No of Faults

Set B= {Bi} i=1 to n where Bi is a Binary String

Representing Faults Covered by Test Case Ti (i=1 to n) .

Each bit corresponds to a Fault. If the Fault is revealed by the test case, the digit is 1; otherwise it is 0.

Output

T'- Prioritized Test Suite

Steps

Set T' to empty

Find S  $\in$  B which is a binary String Bi with large no of 1's.let ti be the test case corresponding to S

T'=T'U {ti}

T=T-{ti}

B=B-{S}

FC=S

While ((no of 1's in FC) <=m)

do

Find Bi which has the maximum Hamming Distance with FC

T'=T'U {ti}

T=T-{ti}

B=B-{Bi}

FC=FC|Bi

Done

Append all remaining test case from T to T'

Print Prioritized Test Suite T'

## V. EXPLANATION WITH AN EXAMPLE

The proposed Algorithm is executed against the example mentioned in Table I.

Inputs

$T = \{t1, t2, t3, t4, t5\}$

$n = 5$

$m = 5$

$B = \{10110, 10100, 11000, 10001, 11100\}$

Initially the proposed algorithm chooses binary string with large number of 1's to select test case with maximum fault coverage. For this example as B1 and B5 both contains maximum of three 1's, B1 is randomly chosen as S.

So at the starting of algorithm  $FC = \{10110\}$  which indicates the faults covered so far by the test suite.

Then at each step, the algorithm chooses  $B_i$  with large Hamming Distance with  $FC$ . Then the test case that corresponds to  $B_i$  is excluded from  $T$  and included in  $T'$ . OR operation is performed between  $FC$  and Selected  $B_i$  to get the faults coverage updated in  $FC$ . This Continues until all faults are covered by test suite (i.e) no of 1's in  $FC$  is equal to the total no of faults  $m$ .

After all faults are covered any remaining test cases in  $T$  is appended to  $T'$  to get the final Prioritized Test Suite.

TABLE II  
VALUES OF VARIABLES IN THE EXECUTION OF PROPOSED  
ALGORITHM FOR EXAMPLE IN TABLE I

| T                                                        | T'               | B                                       | $B_i$     | $t_i$ | $FC = FC \cup B_i$ |
|----------------------------------------------------------|------------------|-----------------------------------------|-----------|-------|--------------------|
| $\{t1, t2, t3, t4, t5\}$                                 | $\{\}$           | $\{10110, 10100, 11000, 10001, 11100\}$ | 1011<br>0 | t1    | 1011<br>0          |
| $\{t2, t3, t4, t5\}$                                     | $\{t1\}$         | $\{10100, 11000, 10001, 11100\}$        | 1100<br>0 | t3    | 1111<br>0          |
| $\{t2, t4, t5\}$                                         | $\{t1, t3\}$     | $\{10100, 10001, 11100\}$               | 1000<br>1 | t4    | 1111<br>1          |
| $\{t2, t5\}$                                             | $\{t1, t3, t4\}$ | $\{10100, 11100\}$                      |           |       |                    |
| Algorithm Stops since no of 1's in $FC$ reached $m (=5)$ |                  |                                         |           |       |                    |
| Append T to T'. Prioritized test Suite                   |                  |                                         |           |       |                    |
| $T' = \{t1, t3, t4, t2, t5\}$                            |                  |                                         |           |       |                    |

## VI. COMPARISON

To quantify the goal of increasing a subset of the test suite's rate of fault detection, a metric called APFD that

Measures the average rate of fault detection per percentage of test suite execution is used. The APFD is calculated by taking the weighted average of the number of faults detected during the run of the test suite. APFD can be calculated using a notation:

Let  $T \rightarrow$  The test suite under evaluation

$m \rightarrow$  the number of faults contained in the program under test  $P$

$n \rightarrow$  The total number of test cases and

$TF_i \rightarrow$  The position of the first test case in  $T$  that exposes fault  $i$ .

$$APFD = 1 - \frac{TF_1 + TF_2 + \dots + TF_m}{nm} + \frac{1}{2n}$$

So as the formula for APFD shows that calculating APFD is only possible when prior knowledge of faults is available. APFD calculations therefore are only used for evaluation.

For the example mentioned in sections IV, the obtained results are compared with the following orderings: No order, Reverse order, Random order and Total Fault Coverage Order of the test cases. The orderings with respect to these approaches are listed in Table III. These approaches are compared by calculating Average Percentage of Faults Detected (APFD) in Tables IV, for the maximum fault coverage concept.

The results obtained for the example explained above have been plotted in APFD charts in Fig.1. These show that the APFD values for the prioritization achieved using the proposed algorithm are better to that obtained with other techniques.

TABLE III  
ORDER OF TEST CASES FOR VARIOUS PRIORITIZATION APPROACHES

| No Order | Reverse Order | Random Order | Total Fault Coverage Order | Proposed order |
|----------|---------------|--------------|----------------------------|----------------|
| t1       | t5            | t3           | t1                         | t1             |
| t2       | t4            | t2           | t5                         | t3             |
| t3       | t3            | t5           | t2                         | t4             |
| t4       | t2            | t4           | t3                         | t2             |
| t5       | t1            | t1           | t4                         | t5             |

TABLE IV  
APFD VALUES OF VARIOUS PRIORITIZATION APPROACHES

| Technique                  | APFD% |
|----------------------------|-------|
| No Order                   | 70%   |
| Reverse Order              | 62%   |
| Random Order               | 58%   |
| Total Fault Coverage Order | 70%   |
| Proposed order             | 82%   |

Fig.2 to Fig.6 show the percentages of faults detected over the fraction of the test suite used, for the orders mentioned in Table III, respectively. The area inside the inscribed rectangles (dashed boxes) represents the weighted percentage of faults detected over the corresponding fraction of the test suite. The solid lines connecting the corners of the inscribed rectangles interpolate the gain in the percentage of detected faults. The area under the curve thus represents the weighted average of the percentage of faults detected over the life of the test suite. Proposed Test order (Fig.6) produces an APFD of 82% and has much faster detecting test order than the test orders obtained using other approaches shown in Fig.2 to Fig.5.

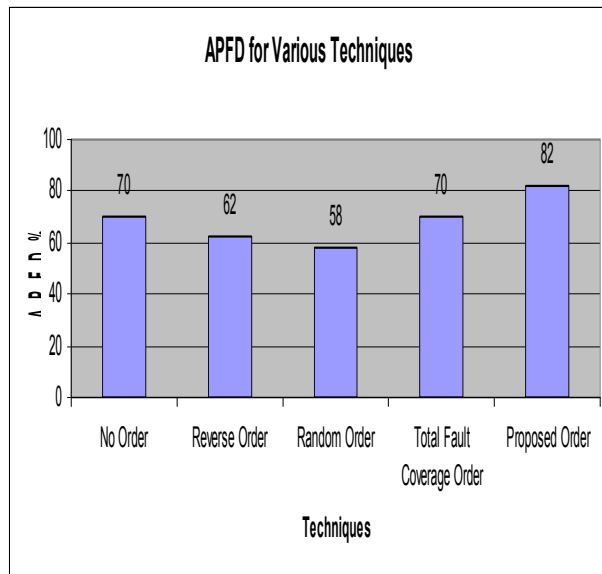


Fig. 1 APFD for Various Prioritization Approaches

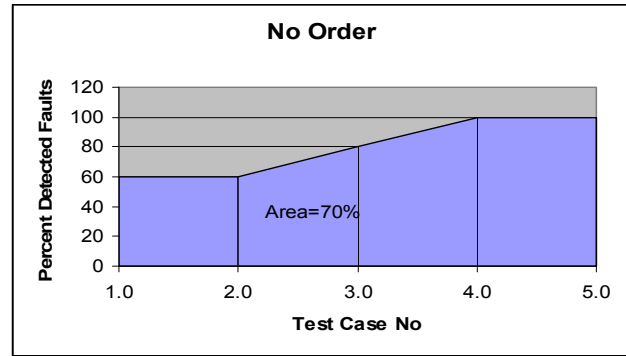


Fig. 2, Percentage of faults detected over the life of the test suite

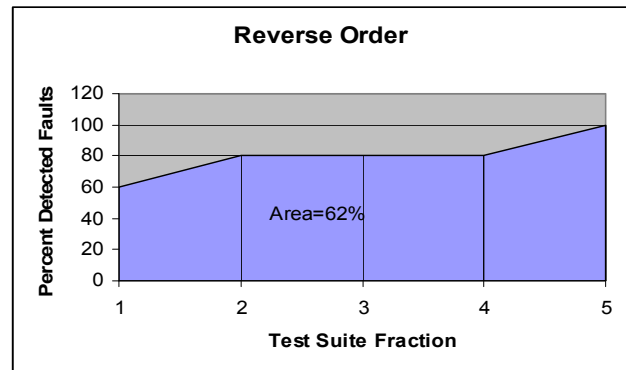


Fig. 3, Percentage of faults detected over the life of the test suite

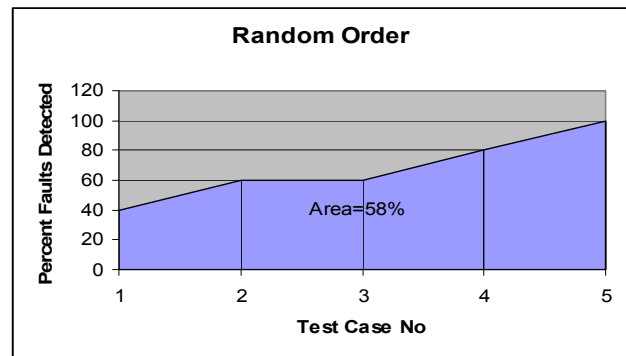


Fig. 4, Percentage of faults detected over the Life of the test suite

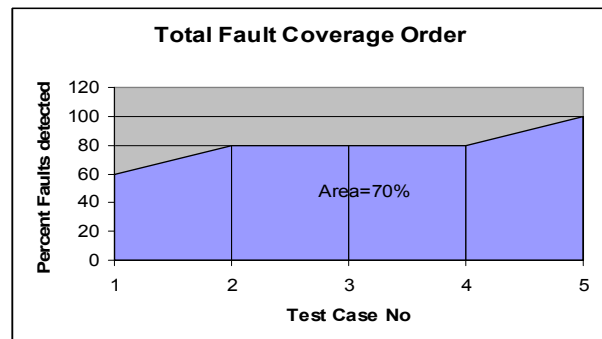


Fig. 5, Percentage of faults detected over the life of the test suite

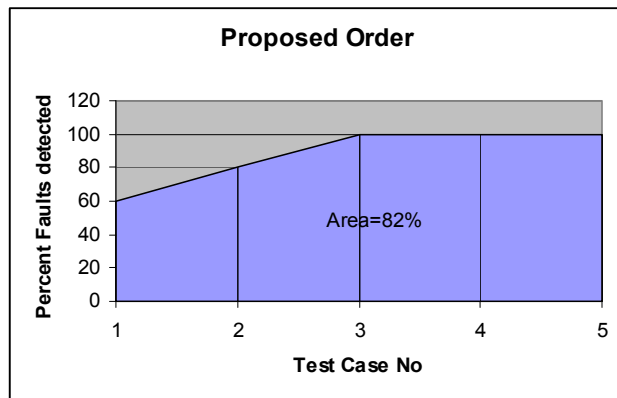


Fig. 6, Percentage of faults detected over the life of the test suite

## VII. CONCLUSION

In this paper, an algorithm is proposed based on Hamming Distance for test case prioritization in order to improve regression testing. Analysis is done for prioritized, non-prioritized, Random and Total Fault Coverage based order cases with the help of APFD (average percentage fault detection) metric. Graphs prove that prioritized case is more effective.

## REFERENCES

- [1] S. Elbaum, A. Malishevsky, and G. Rothermel. "Incorporating varying test costs and fault severities into test case prioritization" In International Conference on Software Engineering, pages 329{338, May 2001.
- [2] S. Elbaum, A. Malishevsky, and G. Rothermel." Test case prioritization: A family of empirical studies", IEEE Transactions of Software Engineering, 28(2):159{182, February 2002.
- [3] J. Jones and M. Harrold. "Test-suite reduction and prioritization for modified condition/decision coverage" In Proceedings of the International Conference on Software Maintenance, Nov. 2001.
- [4] G. Rothermel, R. Untch, C. Chu, and M. Harrold." Test case prioritization" IEEE Transactions on Software Engineering, 27(10):929{948, October 2001.
- [5] S. Elbaum, A. Malishevsky, and G. Rothermel." Test case prioritization: A family of empirical studies. IEEE Transactions of Software Engineering, 28(2):159{182, February 2002.
- [6] G. Rothermel, S. Elbaum, A. Malishevsky, P. Kallakuri, and B. Davia." The impact of test suite granularity on the cost-effectiveness of regression testing". In Proceedings of the 24th International Conference on Software Engineering, pages 230{240, May 2002.
- [7] Srivastava and J. Thiagarajan. "Effectively prioritizing tests in development environment." In Proceedings of the International Symposium on Software Testing and Analysis, pages 97{106, July 2002.
- [8] J.-M. Kim and A. Porter" A history-based test prioritization technique for regression testing in resource constrained environments". In Proceedings of the International Conference on Software Engineering, May 2002.
- [9] Amitabh Srivastava and Jay Islam, M.M., 2012. "MOTCP: A tool or the prioritization of test cases based on a sorting Genetic algorithm and Latent Semantic Indexing. " in Proceedings of the 28th IEEE International Conference on Software Maintenance, Sept. 23-28, IEEE Xplore Press, Trento, pp: 654-657.
- [10] Sabharwal, S., 2011: "A genetic algorithm based approach For prioritization of test case scenarios in static testing. " Proceedings of the 2nd International Conference on computer and Communication Technology, Sept. 15-17, IEEE Xplore Press, Allahabad, pp: 304-309.
- [11] Jastej Badwala and Himanshi Raperia: "Test Case Prioritization using Clustering" International Journal of Current Engineering and Technology, Vol.3, No.2 (June 2013)
- [12] T. Prem Jacob and T. Ravi, "Optimization Of Test Cases By Prioritization, Journal of Computer Science 9 (8): 972-980, 2013